

二叉搜索树

查找：循关键码访问



There's nothing in your head the sorting hat can't see.
So try me on and I will tell you where you ought to be.

邓俊辉

deng@tsinghua.edu.cn

循关键码访问

- ❖ 向量、列表虽各有所长
却不能**兼顾**静态查找与动态修改
- ❖ 为**综合**二者的优点，需改变
对数据集元素的**访问方式**
- ❖ 各数据项依所持**关键码**而彼此区分：**call-by-KEY**
- ❖ 在二叉树中，不妨统一将**data**项作为关键码，它可能
 - 就是整数之类的基本类型，也可能
 - 通过**重载**，指定某成员项作为关键码，并支持**比较与比对**
- ❖ 总之可以假定，关键码的每次比对、比较仅需**常数**时间



基本结构	查找	插入/删除
无序向量	$O(n)$	$O(n)$
有序向量	$O(\log n)$	$O(n)$
无序列表	$O(n)$	$O(1)$
有序列表	$O(n)$	$O(n)$

判等器 + 比较器

```
template <typename T> struct BinNode {  
    T data;  BNP<T> parent, lc, rc; //数据、父节点及左、右孩子  
  
    /* ..... */  
  
    bool operator< ( BinNode const & bn ) { return data <  bn.data; }  
    bool operator<= ( BinNode const & bn ) { return data <= bn.data; }  
    bool operator> ( BinNode const & bn ) { return data >  bn.data; }  
    bool operator>= ( BinNode const & bn ) { return data >= bn.data; }  
    bool operator== ( BinNode const & bn ) { return data == bn.data; }  
  
    /* ..... */  
  
} //BinNode
```

接口

```
template <typename T> class BST : public BinTree<T> { //由BinTree派生
```

```
protected:
```

```
    BNP<T> _x, _p, _r; //查找的终点、其父亲，以及删除之后的替代者
```

```
    BNP<T> connect342( BNP<T>, ..., BNP<T> ); //重构
```

```
    BNP<T> rotate( BNP<T> ); //局部旋转调整
```

```
public: //基本接口: 以virtual修饰，派生的BST变种可默认继承，也可根据各自的规则重写
```

```
    virtual BNP<T> search( const T& ); //查找
```

```
    virtual BNP<T> insert( const T& ); //插入
```

```
    virtual bool remove( const T& ); //删除 (更新高度或不更新)
```

```
}; //BinTree
```

06-A2

二叉搜索树

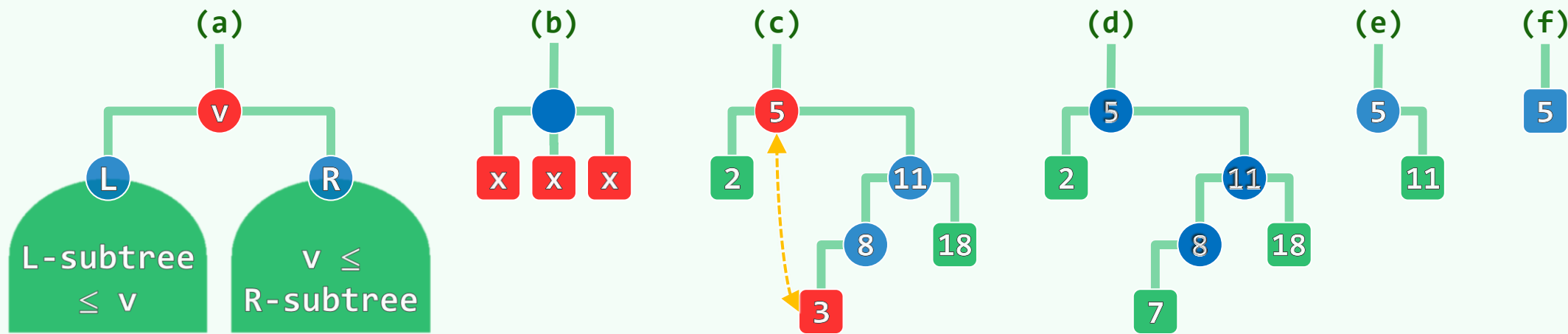
查找：中序

邓俊辉

deng@tsinghua.edu.cn

Good order is the foundation of all good things.

顺序性



❖ 任一节点均

- 不小于其左后代
- 不大于其右后代

❖ 若节点互不相等，则可简化为

- 大于其左后代
- 小于其右后代

相等词条

❖ 为简化起见，暂且

禁止词条（的关键码）**相等**

❖ 当然，这种简化

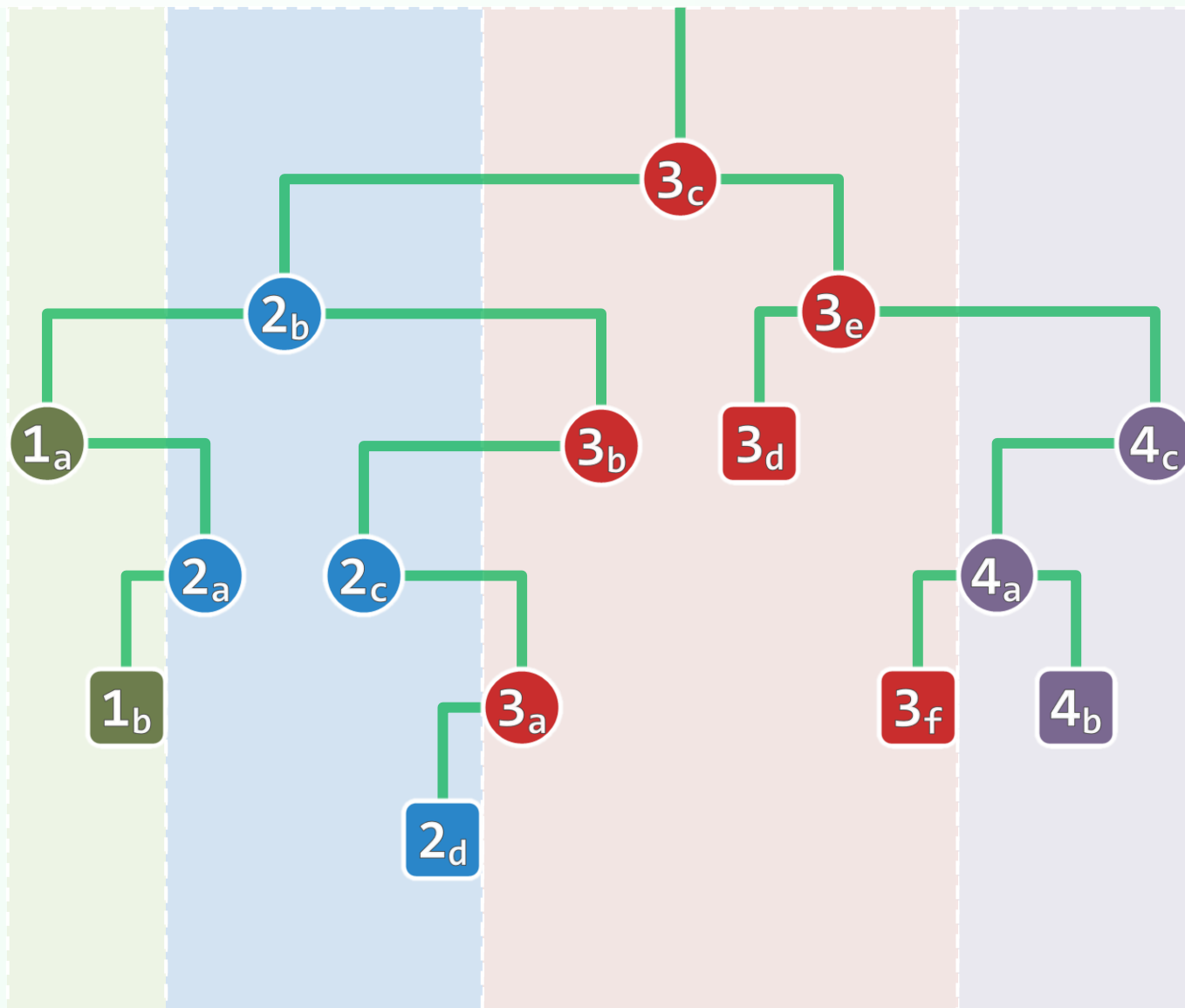
- 在应用中本非自然
- 从算法看亦无必要

❖ 稍后将会看到

相等的关键码完全可以共存：

`searchAll() + searchFirst()`

//习题[7-10|16]、[8-3]



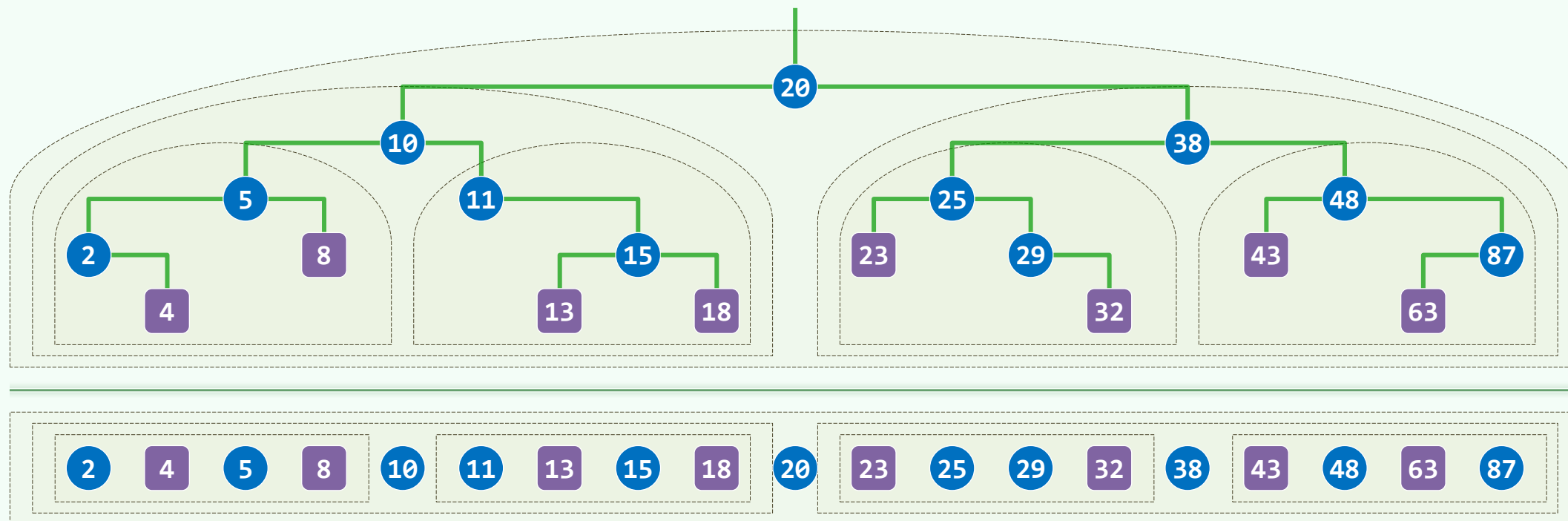
单调性

❖ 顺序性虽只是对局部特征的刻画

却可导出BST整体的单调性

❖ 对树高做数学归纳，不难证明...

BST的中序遍历序列，必然单调非降



06-B1

二叉搜索树

算法及实现：查找

邓俊辉

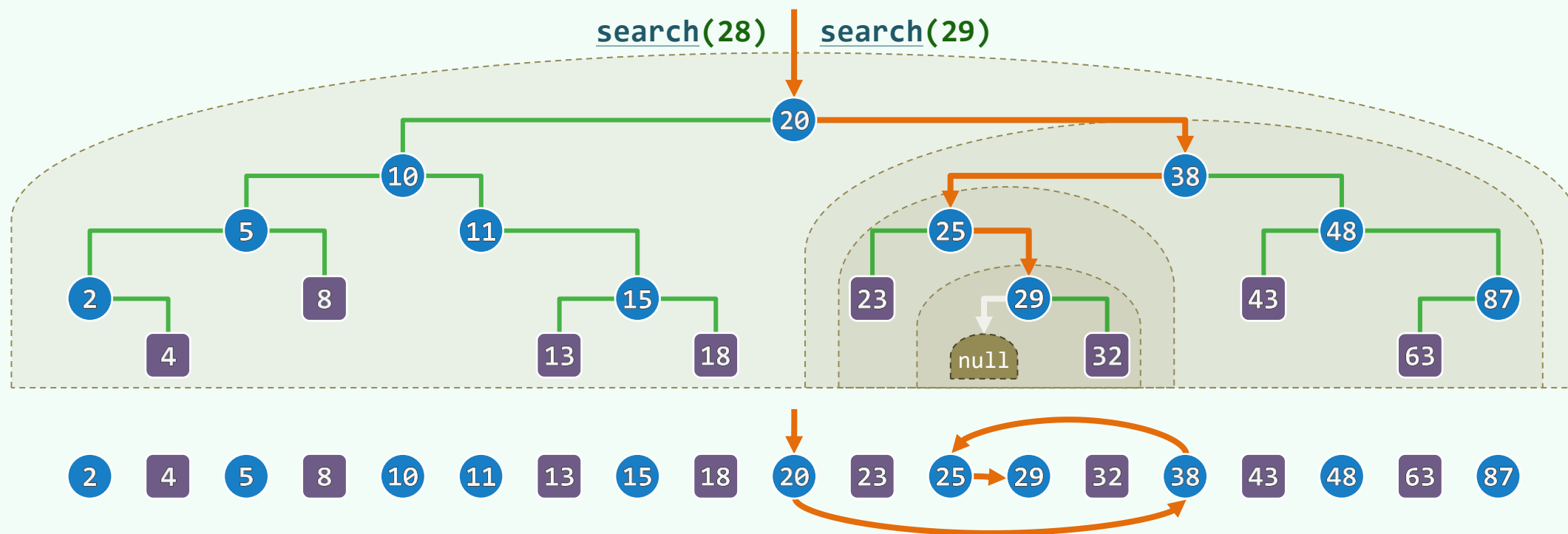
deng@tsinghua.edu.cn

為學日益，為道日損，損之又損，以至於無為，無為而無不為

减而治之

❖ 从根节点出发，逐步地缩小查找范围，直到发现目标（成功），或抵达空树（失败）

❖ 对照中序遍历序列可见，整个过程可视作是在仿效有序向量的二分查找

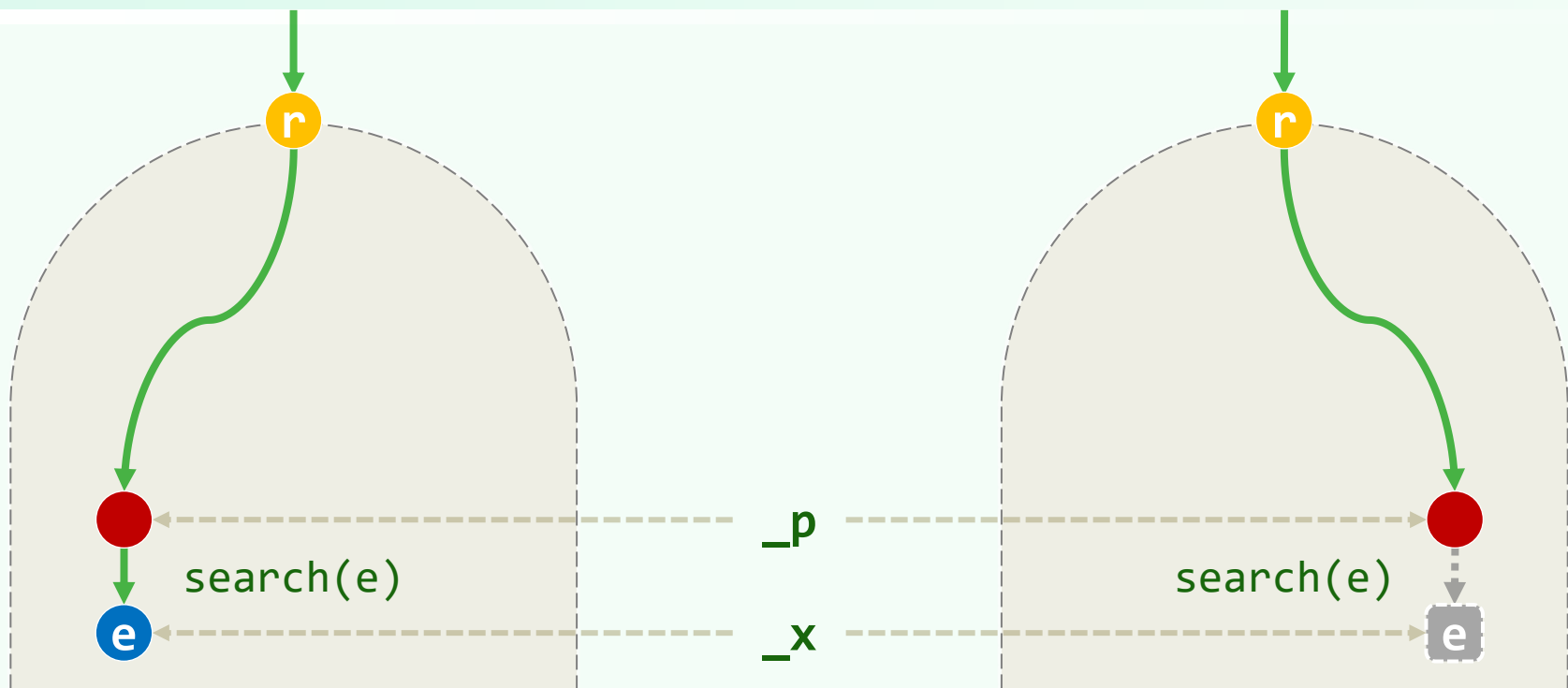


实现

```
template <typename T> BNP<T> BST<T>::search( const T & e ) {  
    for ( _p = NULL, _x = _root; _x && (e != _x->data); ) { //自顶而下  
        _p = _x; // _p紧随_x, 亦步亦趋  
        _x = ( e < _x->data ) ? _x->lc : _x->rc; //向左或右, x逐层深入  
    } //直至_x命中, 或抵达NULL  
    return _x; //无论命中与否, _x为终点, _p为其父  
} //特别地, 命中于树根时, _x == _root且_p == NULL; 树空时, _p == _x == NULL
```

❖ 每一步迭代只需 $O(1)$ 时间 (含判等、比较各一次) , 累计 $O(\text{depth}(_x)) = O(h)$

返回值的语义



❖ $_x$ 指向查找的**终点**: 命中的节点或NULL

$_p$ 指向 $_x$ 的**父亲**: 可能是NULL

❖ 命中于**树根**时, $_x == _root$ 且 $_p == \text{NULL}$

树空时, $_p == _x == \text{NULL}$

06-B2

二叉搜索树

算法及实现：插入

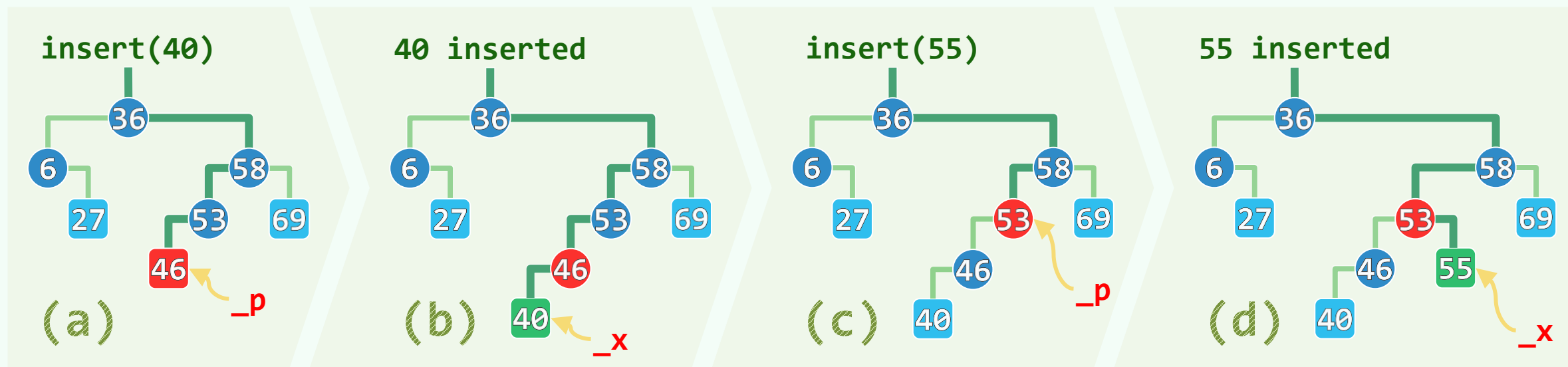
邓俊辉

deng@tsinghua.edu.cn

算法

❖ 先search(e), 确定插入位置_p; 再创建新节点_x, 作为_p的孩子

返回之前，还需更新全树规模及高度



❖ 时间主要消耗于查找与更新高度，均线性正比于_x的深度，不超过树高

实现

```
template <typename T> BNP<T> BST<T>::insert( const T & e ) {  
    if ( search( e ) ) return _p = _x; //有相等元素时，插入无效（用重复的_p和_x标记）  
    _x = new BinNode<T>( e, _p ); //在_x处创建新节点，以_p为父  
    ( _p ? ( *_x < *_p ? _p->lc : _p->rc ) : _root ) = _x; //_p以_x为子：左小右大  
    if ( ( typeid(this) == typeid(BST*) ) && _p ) //更新祖先们的高度  
        _p->updateHeightAbove(); //派生的AVL、红黑树会重写insert()，并自行更新祖先高度  
    _size++; return _x; //新插入的节点，必为叶子  
} //无论e是否存在于原树中，返回时总有_x->data == e
```

二叉搜索树

算法及实现：删除

十步之澤，必有香草；十室之邑，必有忠士

君子無終食之間違仁，造次必於是，顛沛必於是

兄弟，你怎么胡言乱语？世上哪有替死之理？

怎么没有？古人羊角哀、左伯桃舍命全交，名垂青史。小弟要效法古人，替兄受死！

邓俊辉

deng@tsinghua.edu.cn

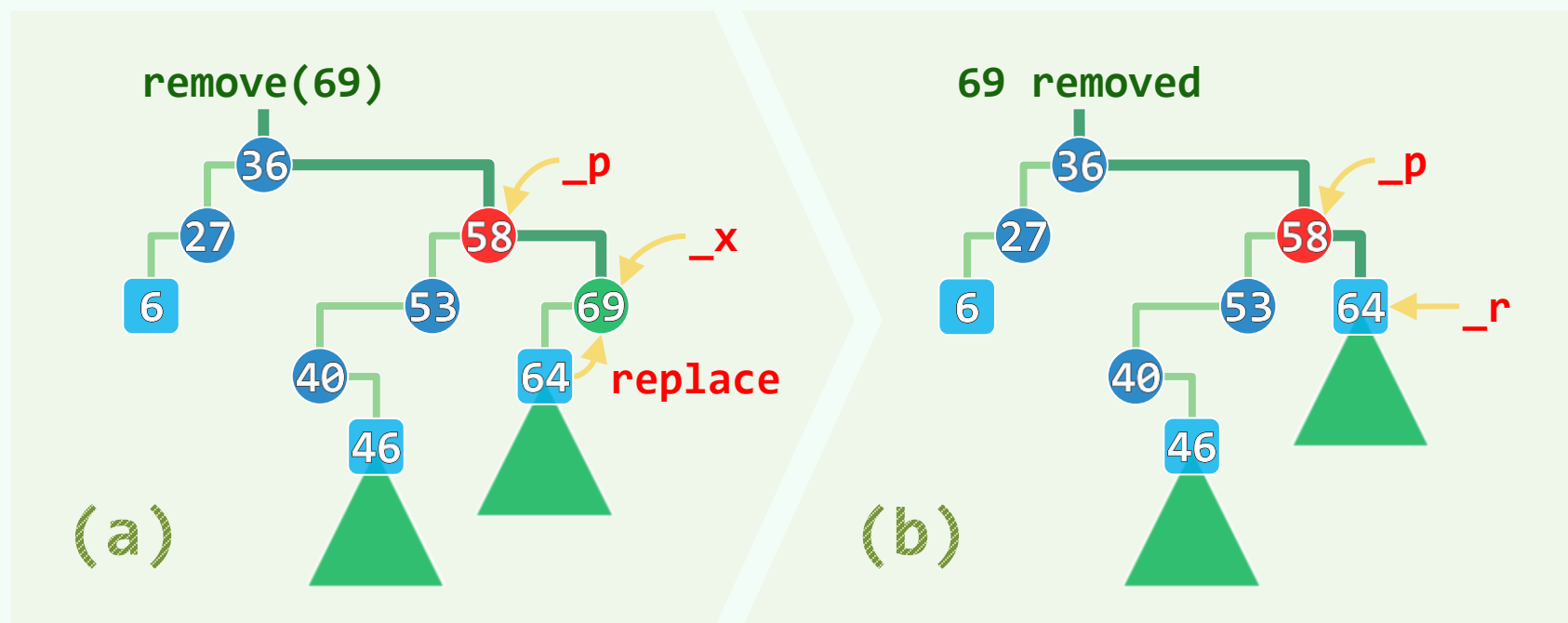
单分支：子承父业

❖ 若 $_x$ (69) 的某一子树为空，则可

将其**替换**为另一子树 (64) //可能亦空

❖ 验证：如此操作之后，二叉搜索树的

拓扑结构依然完整；**顺序性**依然满足



单分支：实现

```
template <typename T> bool BST<T>::remove( const T & e ) {  
    search( e ); if ( !_x ) return false; //确认目标存在 (留意_p的设置)  
  
    /* ... 无论如何，总可以保证_x至多只有一个孩子 ... */  
  
    _r = _x->lc ? _x->lc : _x->rc; //以单子r为接替者 (仍可能是NULL)  
  
    if ( _r ) _r->parent = _p; //_r以_p为父  
  
    ( _p ? ( _p->lc == _x ? _p->lc : _p->rc ) : _root ) = _r; //_p以_r为子  
  
    delete _x; _x = NULL; _size--; //释放_x, 更新全树规模  
  
    if ( (typeid(this) == typeid(BST*)) && _p ) _p->updateHeightAbove();  
  
    return true;  
  
} //remove
```

双分支：李代桃僵

❖ 若：_x (36) 左、右孩子具足

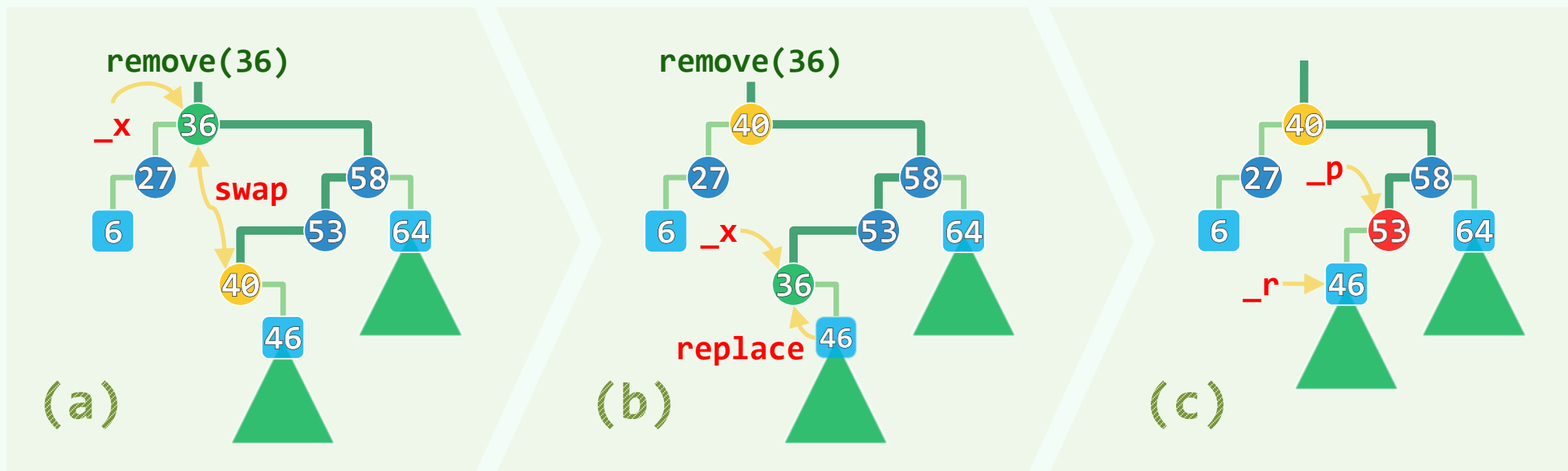
则：找到_x的直接后继 $s = _x \rightarrow \text{succ}()$

交换_x (36) 与s (40, 必无左孩子)

❖ 于是问题**转化**为，照前法删除_x = s

❖ 尽管顺序性在中途曾**一度不合**

但最终必将重新恢复



双分支：实现

```
template <typename T> bool BST<T>::remove( const T & e ) {  
    search( e ); if ( !_x ) return false; //确认目标存在（留意_p的设置）  
  
    if ( _x->lc && _x->rc ) { //双子情况，可转化为单子情况...  
        BNP<T> s = eov( _x->rc ); swap( _x->data, s->data ); //直接后继，s代_x僵  
        _x = s; _p = s->parent; //转移删除操作的焦点  
    } //以下，_x至多只有一个孩子  
    /* ... 照单子情况，统一处理 ... */  
    return true;  
}  
//时间主要消耗于search()、succ()、updateHeightAbove()，累计 $O(\text{depth}(\_x)) = O(h)$ 
```

06-C1

二叉搜索树

平衡：理想与渐近

上梁不正下梁歪

事當難處之時，只讓退一步，便容易處矣
功到將成之候，若放鬆一著，便不能成矣

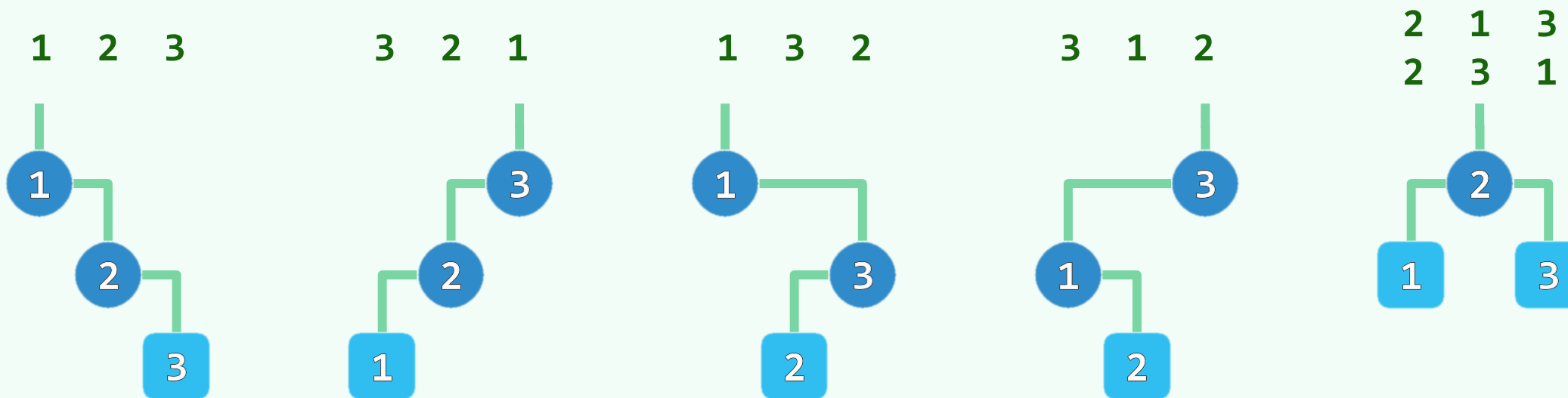
邓俊辉

deng@tsinghua.edu.cn

树高：最坏情况与平均情况

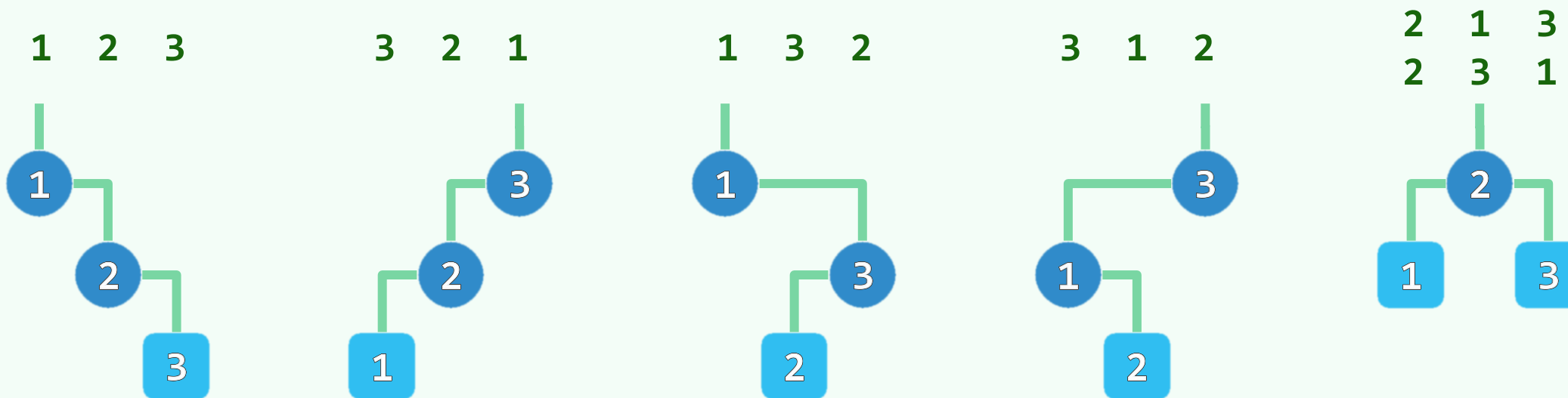
- ❖ 由以上的实现与分析，BST主要接口`search()`、`insert()`和`remove()`的运行时间
在最坏情况下，均线性正比于其高度 $O(h)$
- ❖ 若不能有效地控制树高，就无法体现出BST相对于向量、列表等数据结构的明显优势
- ❖ 比如在最（较）坏情况下，二叉搜索树可能彻底地（接近地）退化为列表
此时的性能不仅没有提高，而且因为结构更为复杂，反而会（在常系数意义上）下降
- ❖ 那么，出现此类最坏、较坏情况的概率有多大？
或者，从平均复杂度的角度看，二叉搜索树的性能究竟如何？
- ❖ 以下按两种常用的随机统计口径，就此做一分析和对比

随机生成：n个词条 $\{ e_1, e_2, e_3, \dots e_n \}$ 按随机排列 $\sigma = (e_{i_1}, e_{i_2}, e_{i_3}, \dots e_{i_n})$ 依次插入



- ❖ 若假设各排列出现的概率**均等** ($1/n!$)，则BST**平均**高度为 $\Theta(\log n)$
- ❖ 的确，多数实际应用中的BST**总体**上都是如此生成和演化的
即便计入remove()，也可通过随机使用succ()和pred()，避免逐渐**倾侧**的趋势
- ❖ 然而问题恰恰在于，所有排列出现的概率的**确均等**吗？

随机组成：n个互异节点，在遵守顺序性的前提下，随机确定代数的联接关系



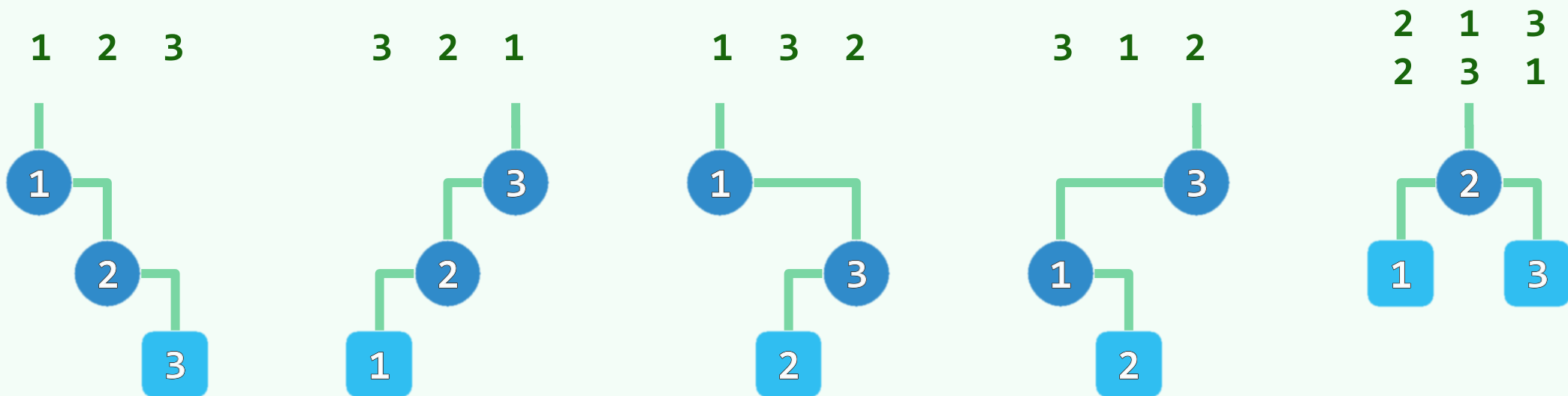
❖ 由n个互异节点随机**组成**的BST，若共计 $S(n)$ 棵，则有

$$S(n) = \sum_{k=1}^n S(k-1) \cdot S(n-k) = \text{catalan}(n) = (2n)! / (n+1)! / n!$$

❖ 假定所有BST**等概率**地出现，则其**平均**高度为 $\Theta(\sqrt{n})$

❖ 在Huffman编码之类的应用中，二叉树（尽管还不是BST）的确是逐渐**拼合**而成的

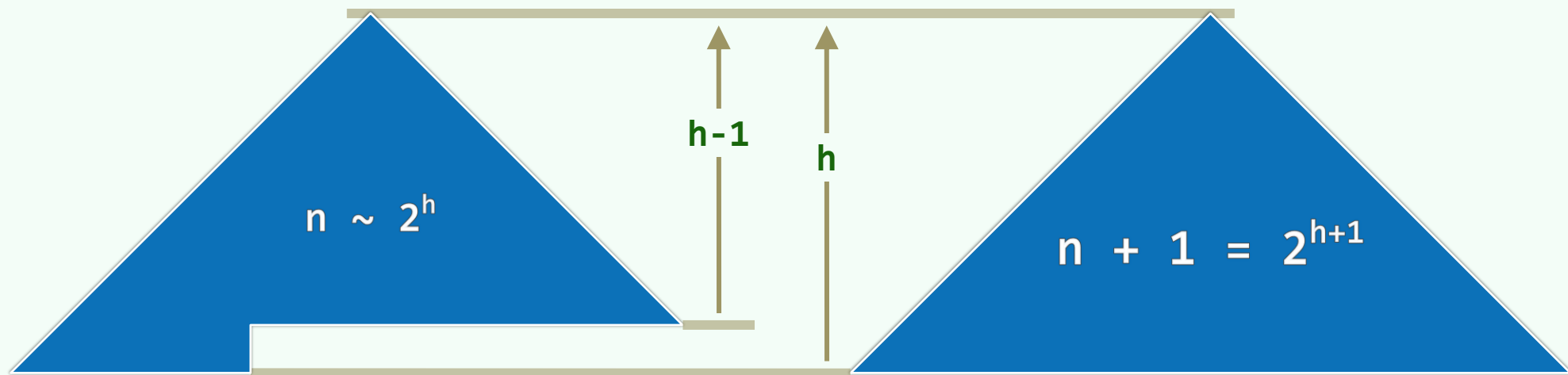
$\log n$ vs. $\sqrt{n}$



- ❖ 两种口径所估计出的平均高度差异极大——谁更可信？谁更接近于真实情况？
- ❖ 后者未免太**悲观**，但前者则过于**乐观**：BST越低，**权重**越大；而最根本的原因在于...
- ❖ **理想随机**在实际中**绝难出现**：局部性、关联性、（分段）单调性、（近似）周期性、...
较高甚至极高的BST频繁出现，不足为怪；平衡化处理**很有必要**！

理想平衡

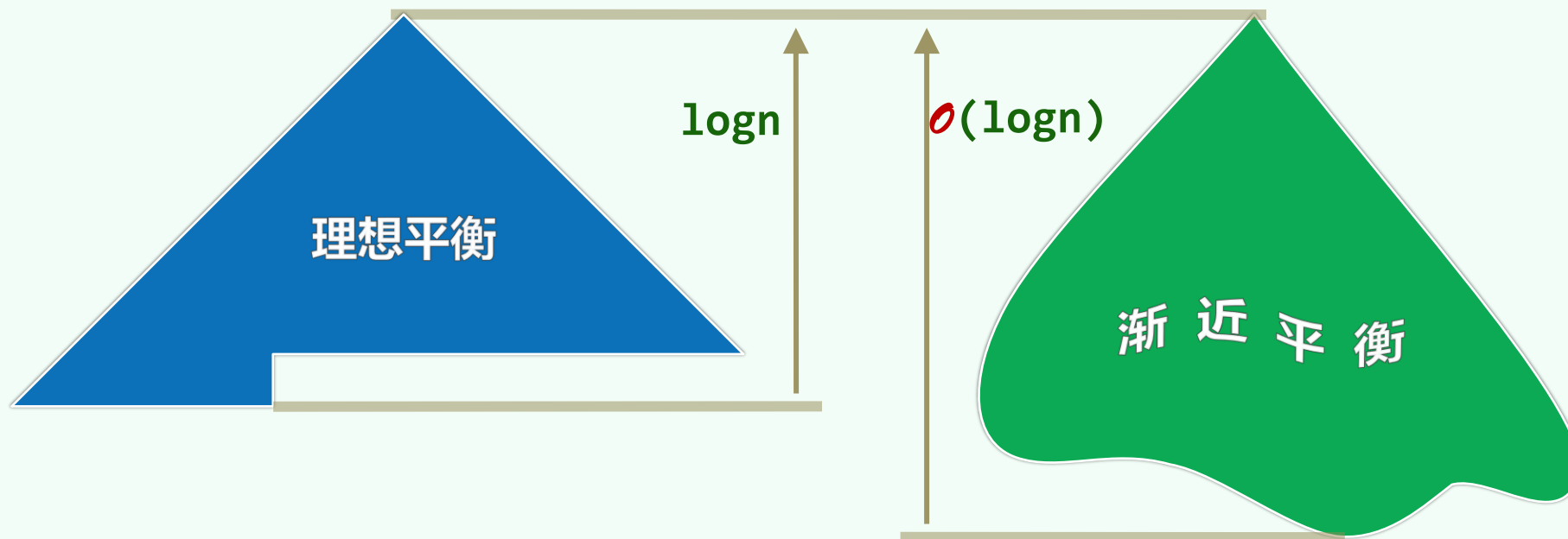
- ❖ 节点数目固定时，兄弟子树的高度越接近（平衡），全树也将倾向于更低
- ❖ 由 n 个节点组成的二叉树，高度不致低于 $\lfloor \log_2 n \rfloor$ ——达到这一下界时，称作理想平衡



- ❖ 大致相当于完全树甚至满树：叶节点只能出现于最底部的两层——条件过于苛刻

渐近平衡

- ❖ 理想平衡出现的概率太低，可遇不可求；成本太高，更难以为继
- ❖ 退一步海阔天空：高度渐近地不超过 $\mathcal{O}(\log n)$ ，即可接受



- ❖ 渐近平衡的BST，即视作平衡二叉搜索树 | balanced BST | BBST

06-C2

二叉搜索树

平衡：等价变换

(清華) 校內無上下尊卑之分，當有長幼先後之序
—— 周詒春

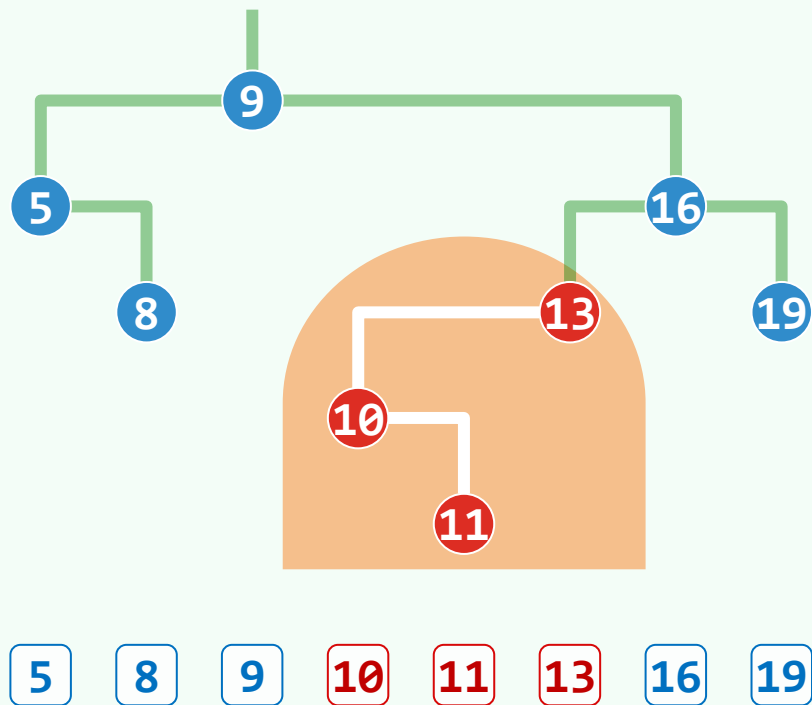
邓俊辉

deng@tsinghua.edu.cn

中序序列的歧义 ~ BST的等价

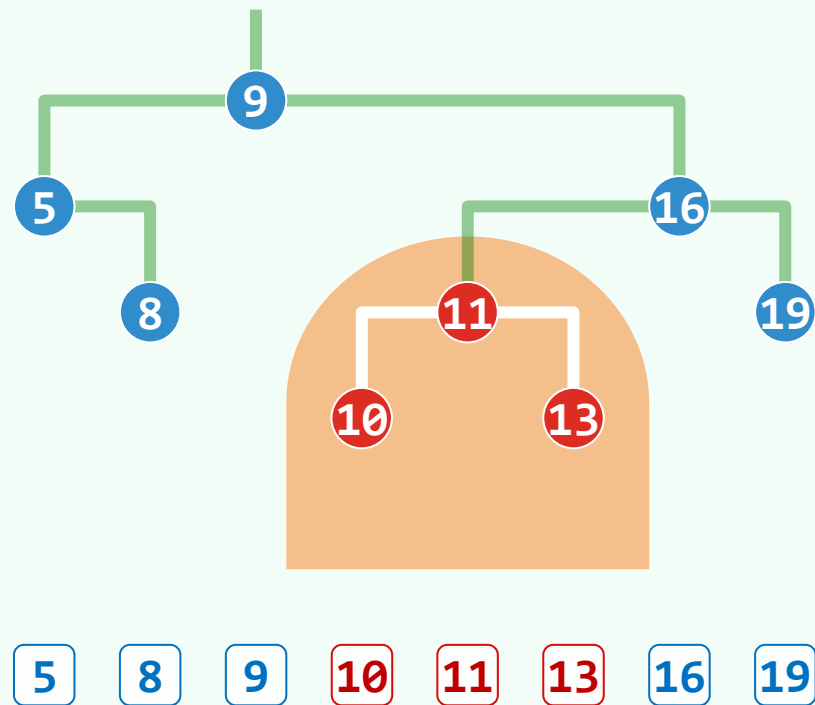
❖ 上下可变

- 联接关系不尽相同
- 承袭关系可能颠倒



❖ 左右不乱 // 你学长永远是你学长

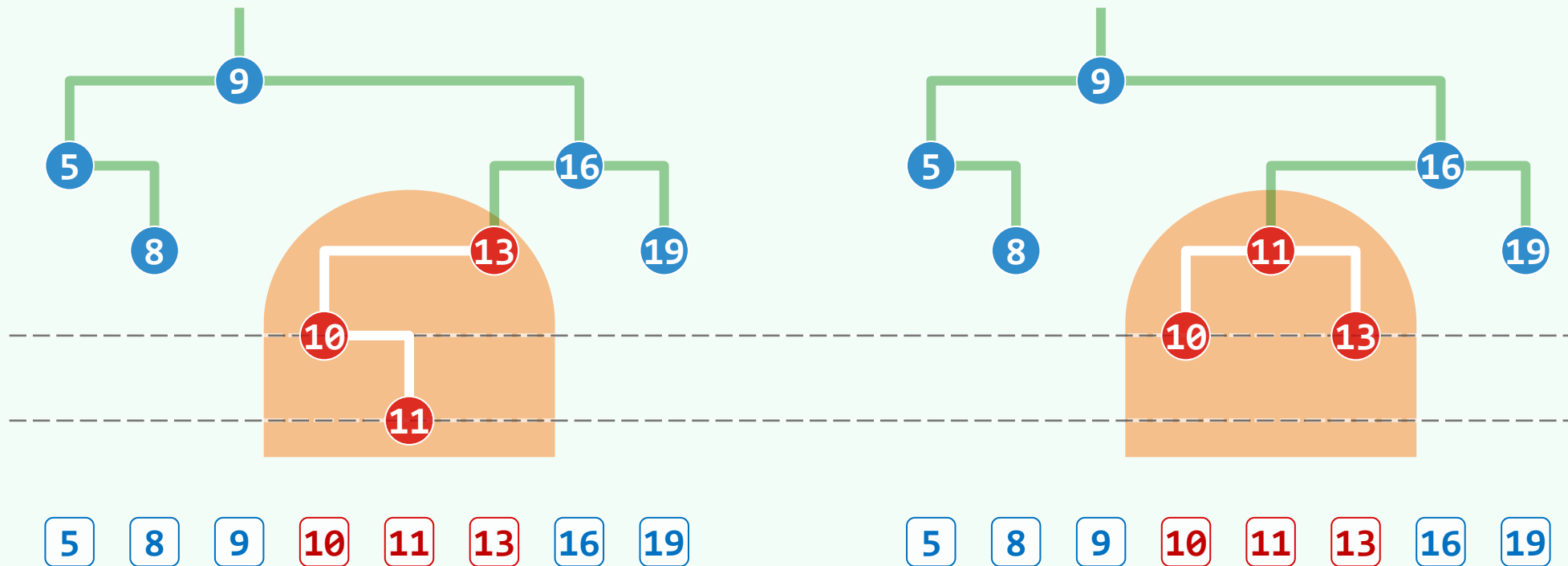
- 中序遍历序列完全一致
- 全局始终单调非降



限制条件：违反+修复

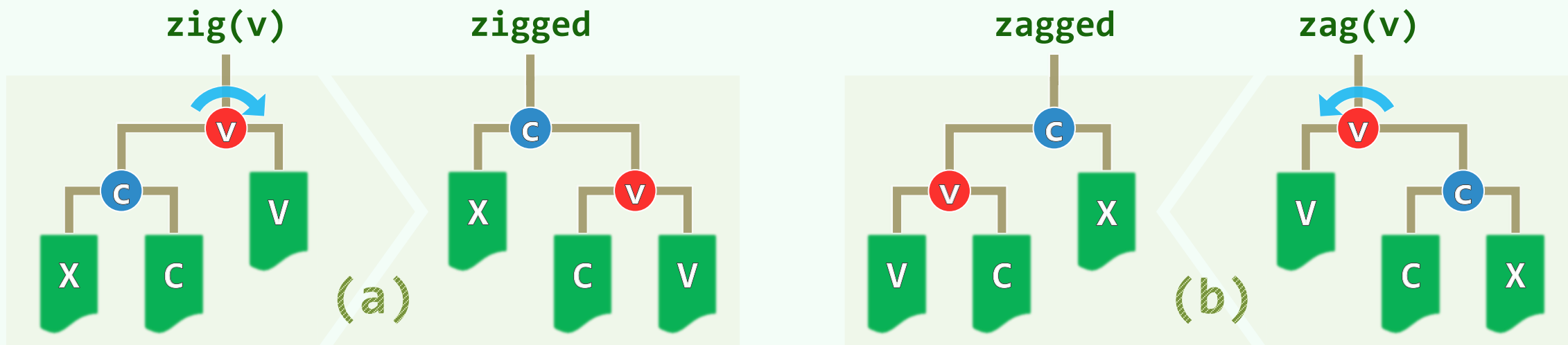
❖ BBST有众多变种，均各自约定了一套**限制条件**

- 在一次ADT操作之后，虽然可能至多有 $\mathcal{O}(\log n)$ 处局部（同时或相继地）**违反限制条件**
- 但好在，总是可以在 $\mathcal{O}(\log n)$ 时间内予以**修复**



等价变换 + 旋转调整：序齿不序爵

❖ 刚刚失衡的BST，必可**迅速**转换为一棵等价的BBST——为此，只需 $\mathcal{O}(\log n)$ 甚至 $\mathcal{O}(1)$ 次旋转



❖ zig和zag：仅涉及**常数**个节点，只需调整其间的联接关系；均属于**局部的基本**操作

❖ 调整之后：v/c深度加/减1，子（全）树高度的变化幅度，上下差异**不超过1**

❖ 实际上，经过不超过 $\mathcal{O}(n)$ 次旋转，等价的BST均可相互转化（习题解析[7-15]）

06-D1

二叉搜索树

AVL树：渐近平衡

The maxim "Nothing avails but perfection" may be
spelt shorter: "Paralysis."

邓俊辉

deng@tsinghua.edu.cn

平衡因子

❖ 平衡的度量

balance factor

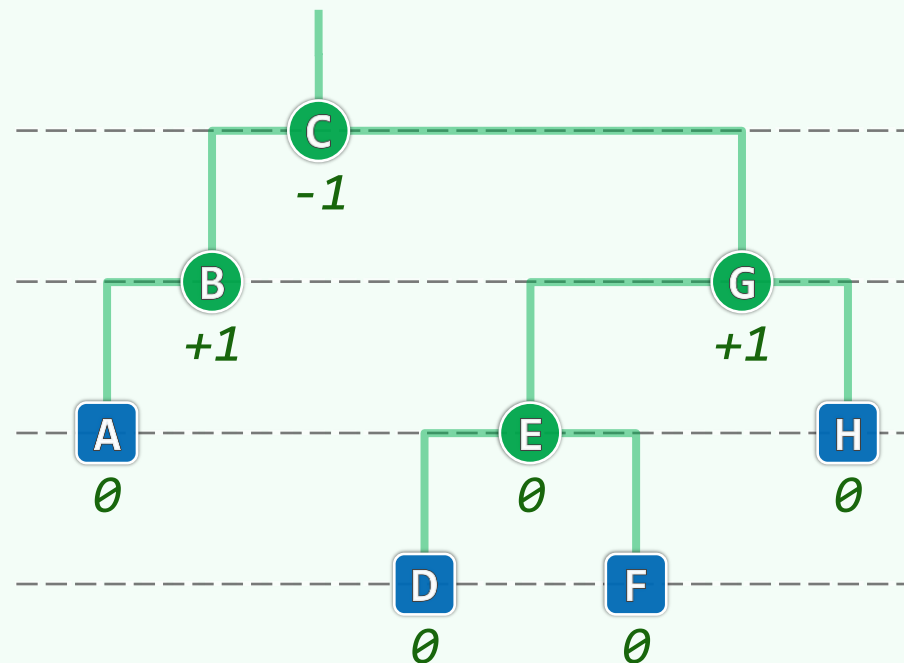
$$balFac(v) = height(lc(v)) - height(rc(v))$$

❖ 平衡的标准

G. Adelson-Velsky & E. Landis (1962)

$$\forall v \in AVL, |balFac(v)| \leq 1$$

❖ AVL树未必理想平衡，但总是渐近平衡...



```

#define BalFac(x)  ( Height( (x)->lc ) - Height( (x)->rc ) ) //平衡因子

#define AvlBalanced(x)  ( ( -2 < BalFac(x) ) && ( BalFac(x) < 2 ) ) //AVL平衡条件

template <typename T> class AVL : public BST<T> { //由BST派生
    public:

        BNP<T> insert( const T& ); //插入 (重写)

        bool remove( const T& ); //删除 (重写)

        //BST::search()等接口, 可直接沿用

}; //AVL

```

AVL = 渐近平衡

❖ 由 n 个节点组成的AVL树，高度 $h = \mathcal{O}(\log n)$ ，或等价地

高度为 h 的AVL树，规模 $n = \Omega(\phi^h)$

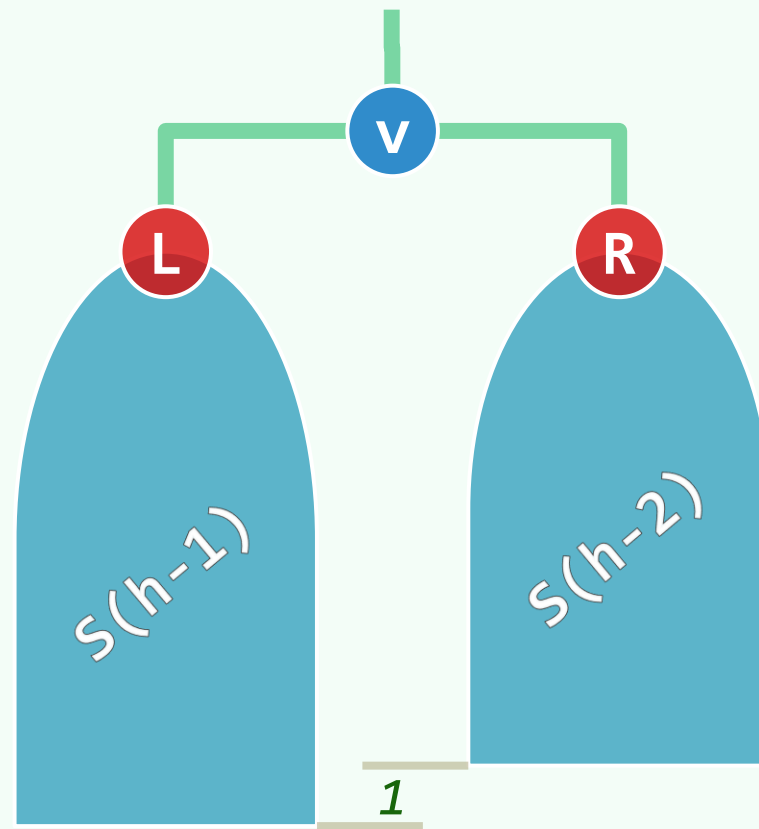
❖ 固定高度 h ，考查节点**最少**的AVL树

将其规模记作 $S(h)$

❖ 于是： $S(h) = 1 + S(h-1) + S(h-2)$

❖ 递推： $S(h) + 1 = [S(h-1) + 1] + [S(h-2) + 1]$
 $= \dots = fib(h+3)$

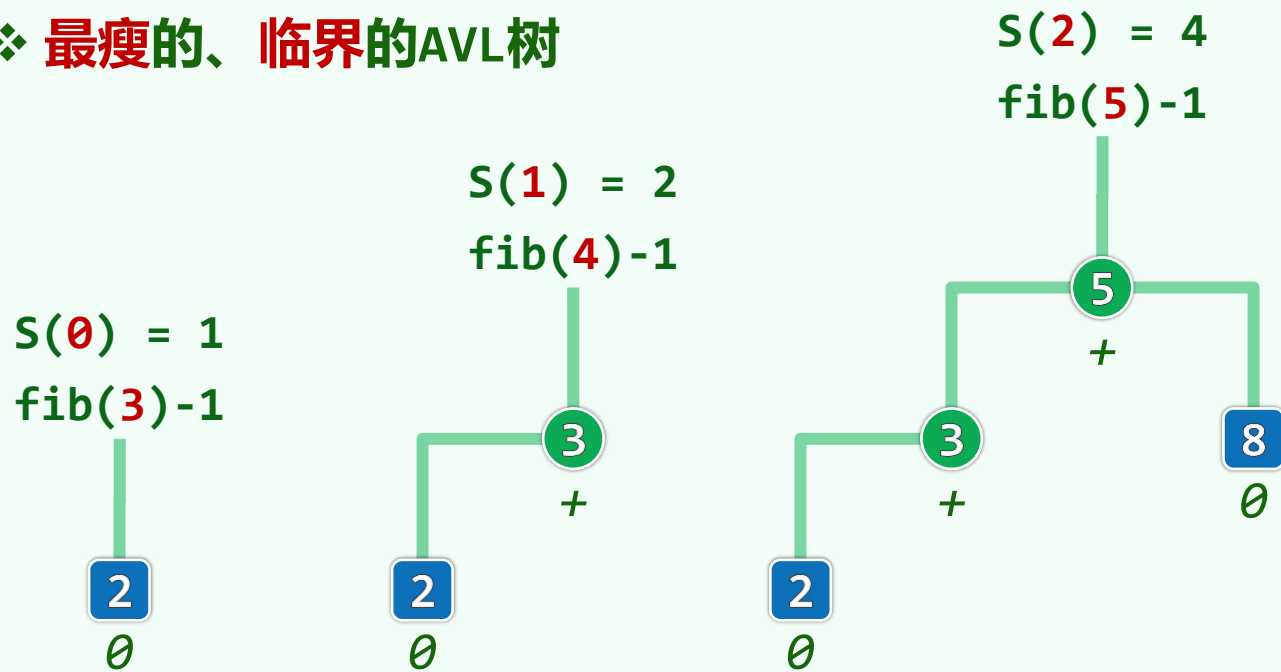
❖ 亦即： $S(h) = fib(h+3) - 1 = \Omega(\phi^h)$



Fibonacci Tree

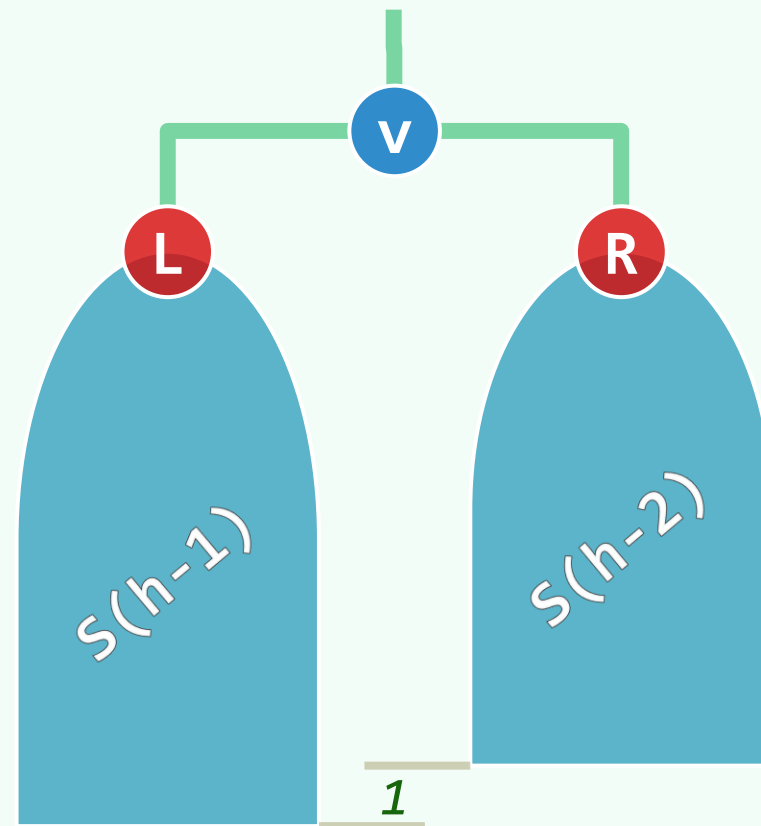
❖ 高度为 h , 规模恰为 $S(h) = \text{fib}(h + 3) - 1$ 的AVL树

❖ 最瘦的、临界的AVL树



1, 2, 4, 7, 12, 20, 33, 54, 88, 143, ...

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, ...



06-D2

二叉搜索树

AVL树：失衡与复衡

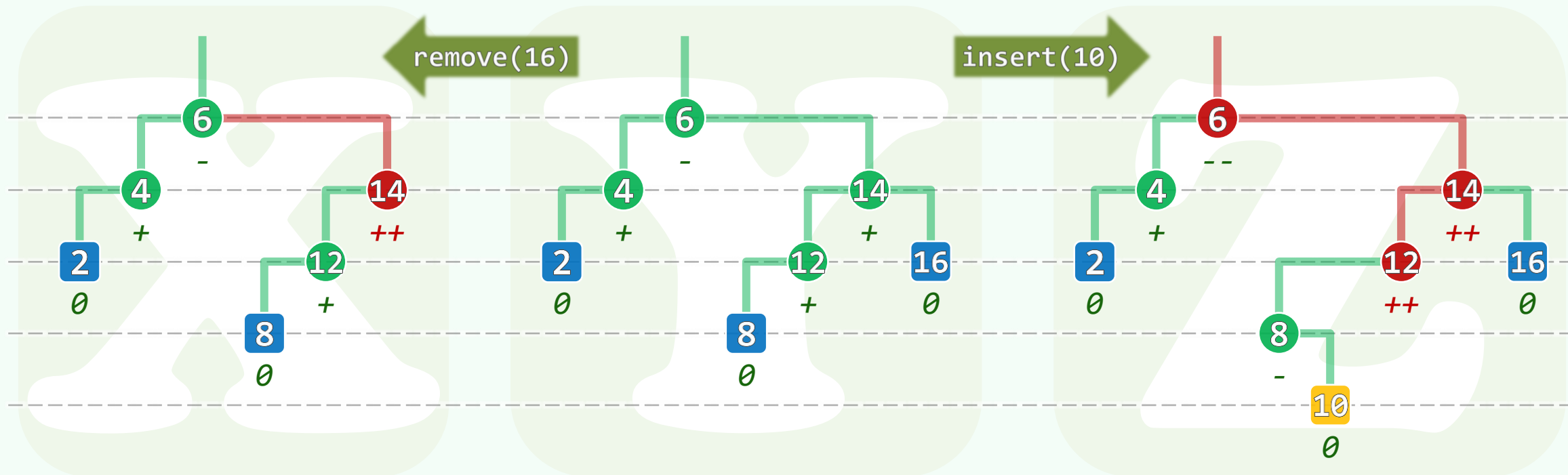
邓俊辉

deng@tsinghua.edu.cn

不取於相，如如不動

失衡

❖ 按BST规则动态操作之后，AVL平衡性可能破坏 //当然，只涉及到祖先

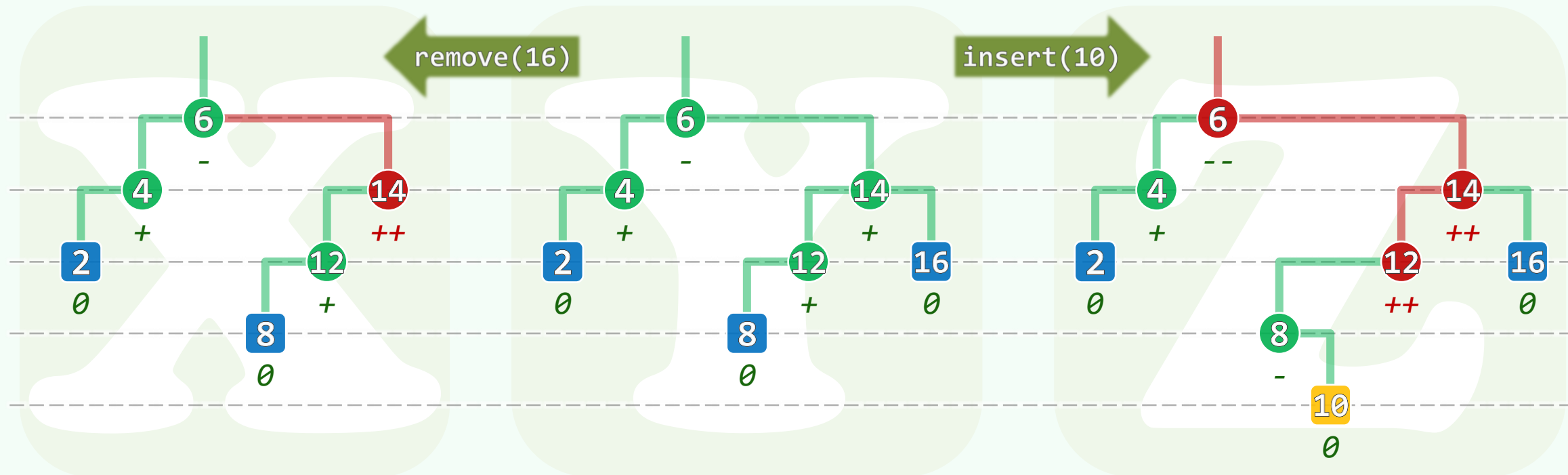


❖ 插入：从祖父开始，每个祖先都有可能失衡，且可能同时失衡！ //复杂？

❖ 删除：从父亲开始，每个祖先都有可能失衡，但任一时刻至多一个！为什么？ //简单？

重平衡

❖ 如何恢复平衡？蛮力不足取，须借助**等价变换**！



❖ 局部性：所有的旋转都在**局部**进行 //每次只需 $O(1)$ 时间

快速性：在每一**深度**只需检查并旋转至多一次 //共 $O(\log n)$ 次

06-D3

二叉搜索树

AVL树：插入

邓俊辉

deng@tsinghua.edu.cn

名不正則言不順，言不順則事不成

单旋：黄节点恰好存在其一

❖ 同时可有多个失衡节点

最低者g不低于_x的祖父

❖ 逐层上溯，便可找到g

❖ 确定名分：

- $p = \text{tallerChild}(g)$

- $v = \text{tallerChild}(p)$

❖ 无论p和v的方向是否一致

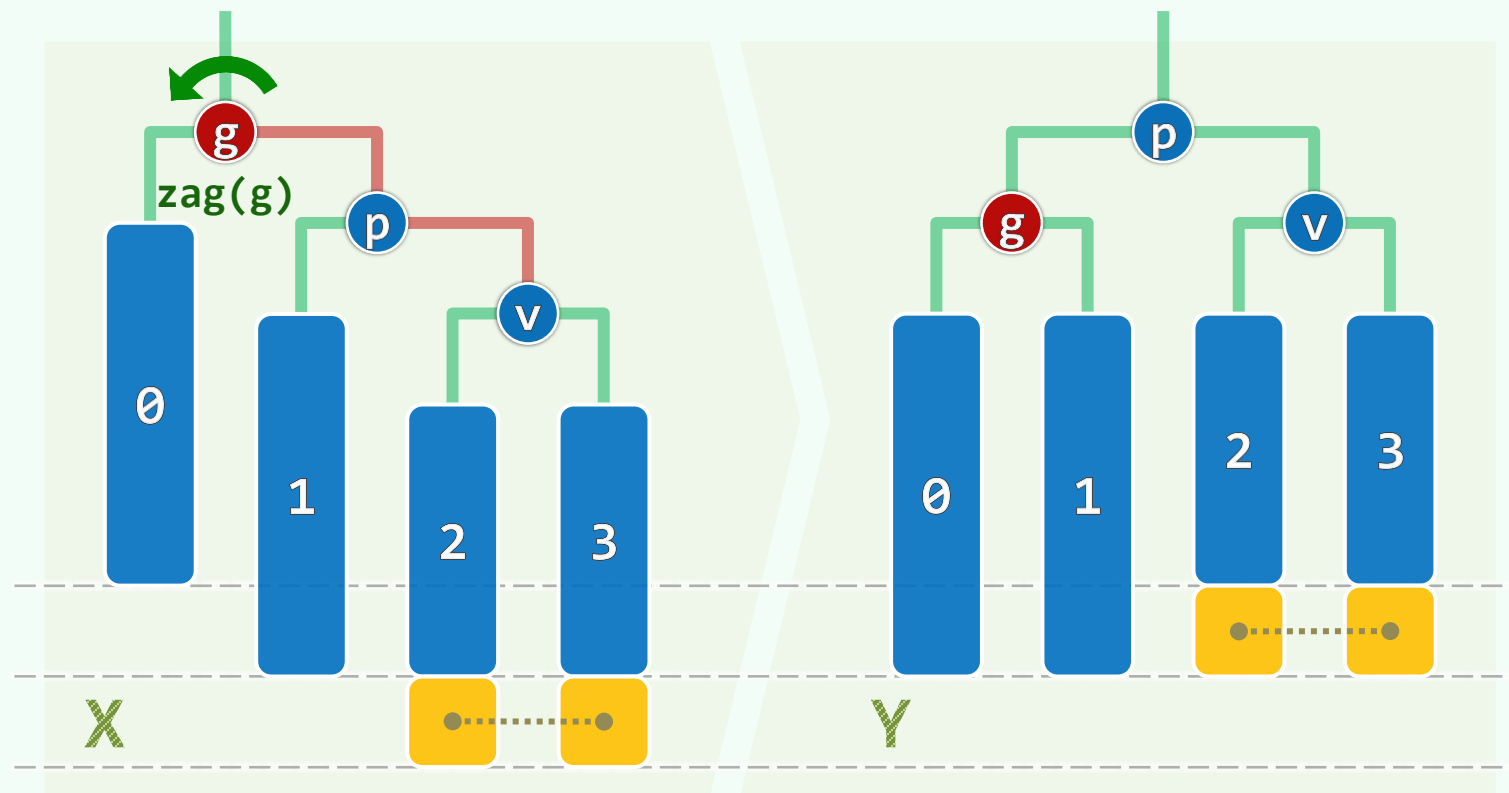
均可从容处理...

❖ g经单旋调整后复衡

子树高度复原

❖ 更高祖先也必平衡

全树复衡



双旋

❖ 同时可有多个失衡节点

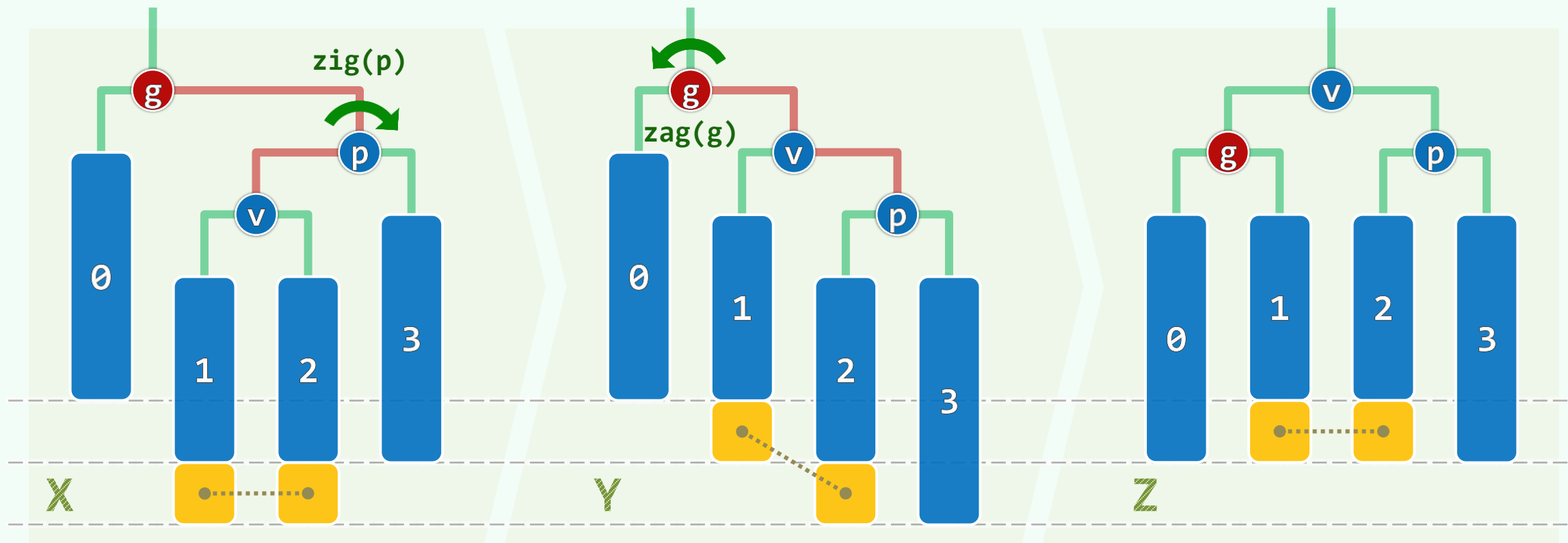
最低者g不低于_x的祖父

❖ g经单旋调整后复衡

子树高度复原

❖ 更高祖先也必平衡

全树复衡



实现

```
template <typename T> BNP<T> AVL<T>::insert( const T & e ) {  
    BST::insert( e ); if ( _p == _x ) return _x; //先按BST规则插入  
    for ( BNP<T> g = _p; g; g = g->parent ) { //逐层上溯历代祖先g  
        if ( !AvlBalanced( g ) ) g = rotate( g ); //一旦g失衡, 则经旋转使之复衡  
        g->updateHeight(); //更新子树高度  
        if ( 0 == BalFac( g ) ) break; //提前终止上溯  
    } //局部子树复衡后, 高度必然复原; 所有祖先亦必复衡: 累计至多会做 $O(1)$ 次调整  
    return _x; //插入成功  
} //insert
```

06-D4

二叉搜索树

AVL树：删除

邓俊辉

deng@tsinghua.edu.cn

没有什么是一次旋转解决不了的；如果有，那就两次

单旋：黄节点至少存在其一；红节点可有可无

❖ 瞬时至多一个失衡节点 g ，
可能就是 $_x$ 的父亲 $_p$

❖ 复衡后子树高度未必复原
更高祖先仍可能随之失衡

❖ 失衡可能持续向上传播
最多需做 $O(\log n)$ 次调整

❖ 逐层上溯，便可找到 g

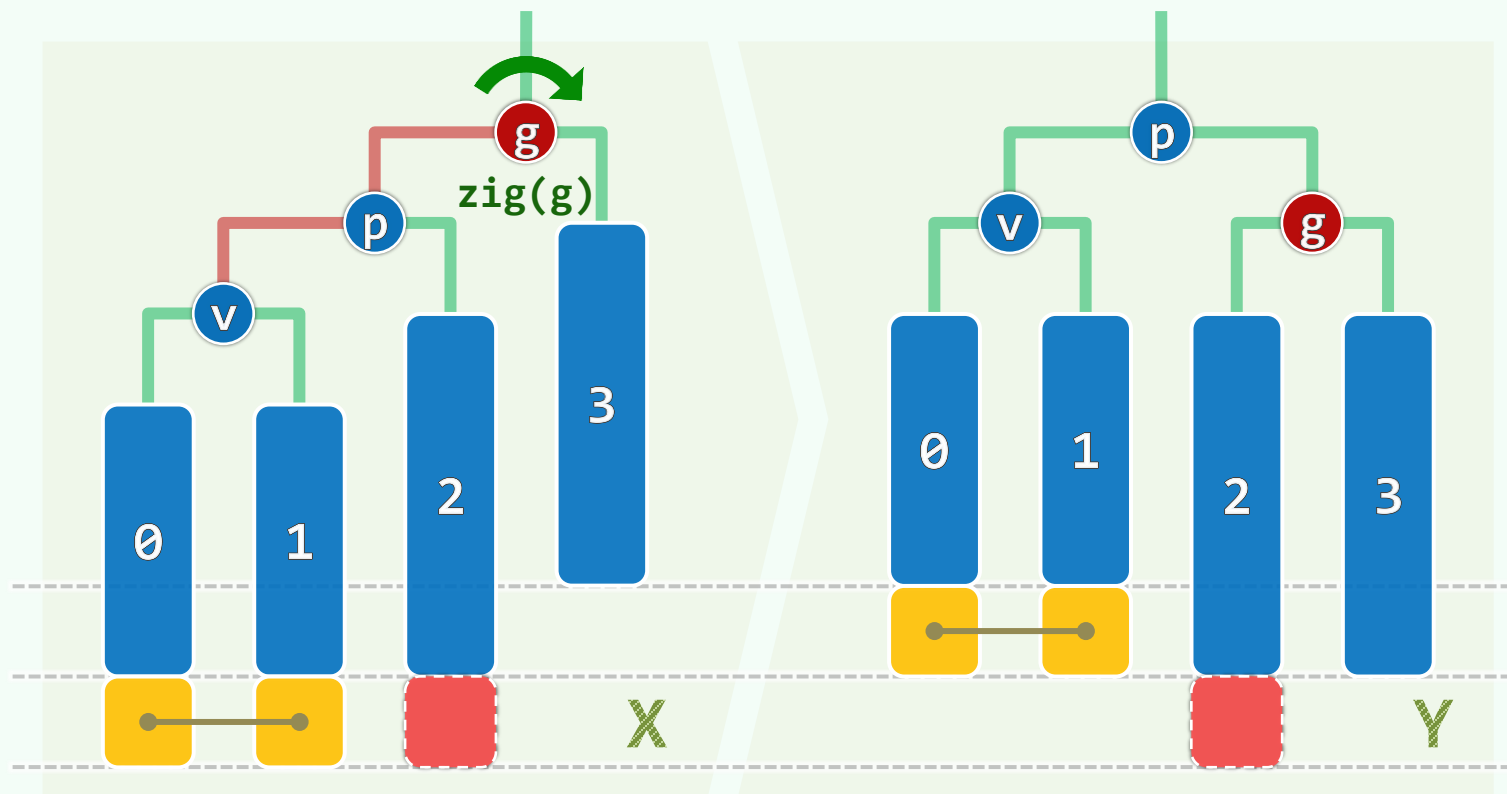
❖ 确定名分：

- $p = \text{tallerChild}(g)$

- $v = \text{tallerChild}(p)$

// 红节点存在时，有歧义？

❖ 无论 p 和 v 的方向是否一致
均可从容处理...

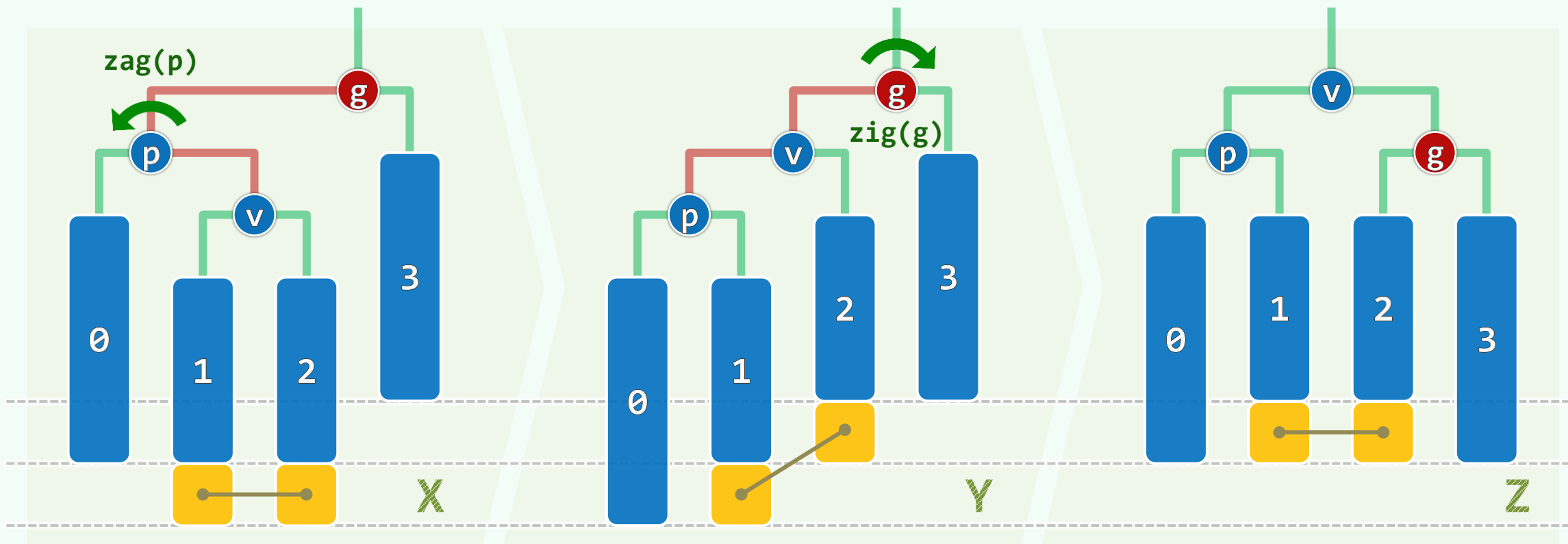


双旋

❖ 瞬时**至多一个**失衡节点 g ,
可能就是 $_x$ 的父亲 $_p$

❖ 复衡后子树高度**不能**复原
更高祖先仍可能**随之**失衡

❖ 失衡**可能**持续向上传播
最多需做 $O(\log n)$ 次调整



实现

```
template <typename T> bool AVL<T>::remove( const T & e ) {  
    if ( !BST::remove( e ); ) return false; //先按BST规则删除  
    for ( BNP<T> g = _p; g; g = g->parent ) { //逐层上溯历代祖先g  
        if ( !AvlBalanced( g ) ) g = rotate( g ); //每当g失衡, 都经旋转使之复衡  
        g->updateHeight(); //更新子树高度  
        if ( 0 != BalFac( g ) ) break; //提前终止上溯  
    } //局部子树复衡后, 高度未必复原; 更高祖先仍或失衡: 累计可能需做 $\Omega(\log n)$ 次调整  
    return true; //删除成功  
} //remove
```

06-D5

二叉搜索树

AVL树: (3+4-2)-重构

邓俊辉

deng@tsinghua.edu.cn

為學之道至簡至易，但患不知其方

返璞归真

- ❖ 纵观AVL树重平衡的各种情况，无非就是局部的**三个节点**、**四棵子树**按照**中序**统一**重构**...

- ❖ 三个节点无论高低，统一按**中序**命名为：

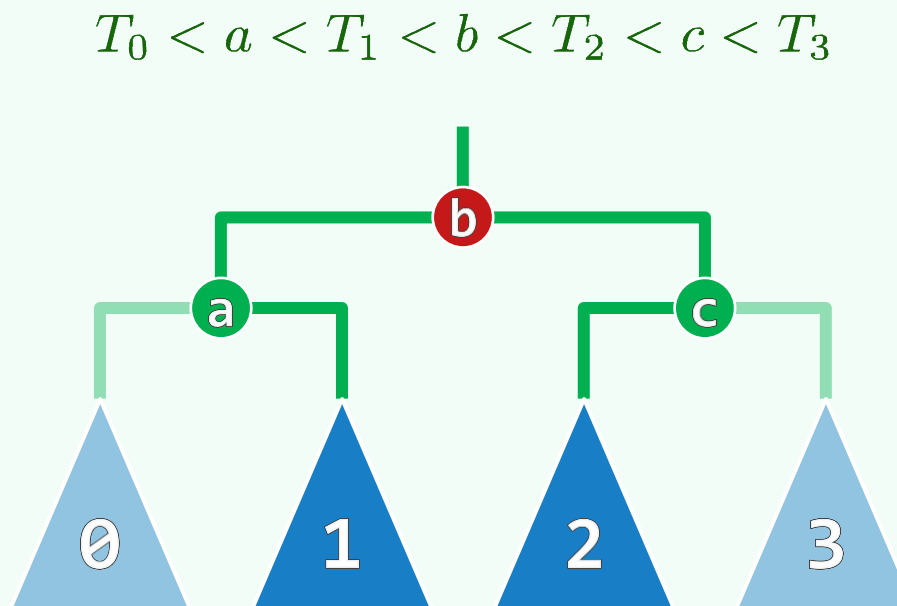
$$a < b < c$$

- 四棵子树无论大小，亦是按**中序**命名为：

$$T_0 < T_1 < T_2 < T_3$$

- ❖ 所谓的重平衡，无非就是...

- ❖ ...将原先的子树**g**，替换为**等价**的子树**b**



- ❖ 进一步地， T_0 与 a 、 c 与 T_3 的联接**无需**调整！

(3+4-2)-重构

```
template <typename T> BST<T>::connect342
```

```
( BNP<T> a, BNP<T> T1, BNP<T> b, BNP<T> T2, BNP<T> c ) {
```

```
    b->attachLc( a );
```

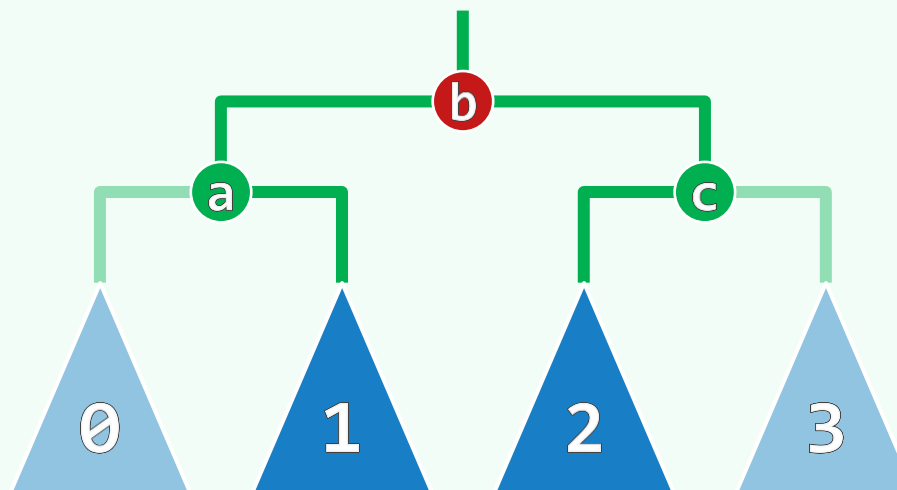
```
    a->attachRc( T1 ); a->updateHeight();
```

```
    b->attachRc( c );
```

```
    c->attachLc( T2 ); c->updateHeight();
```

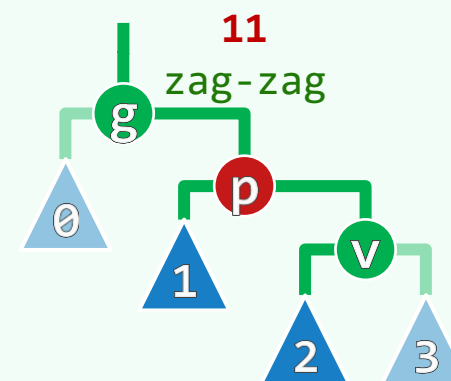
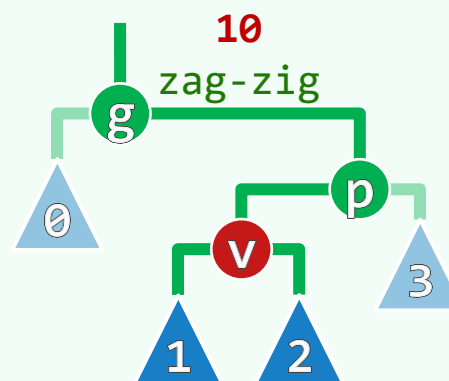
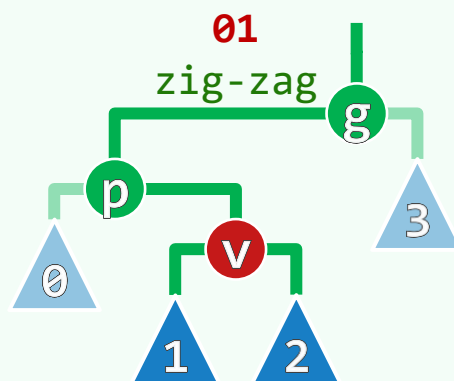
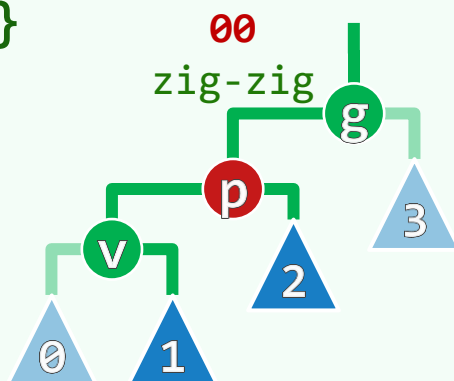
```
    return b; //该子树新的根节点，其高度由调用者负责更新
```

```
} //connect342
```



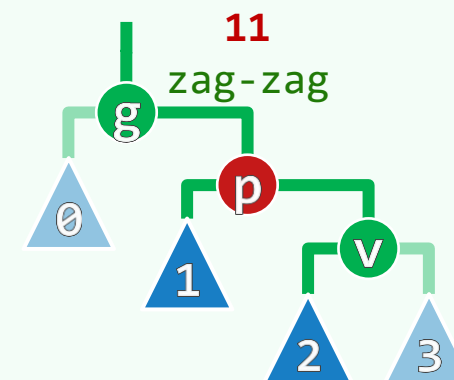
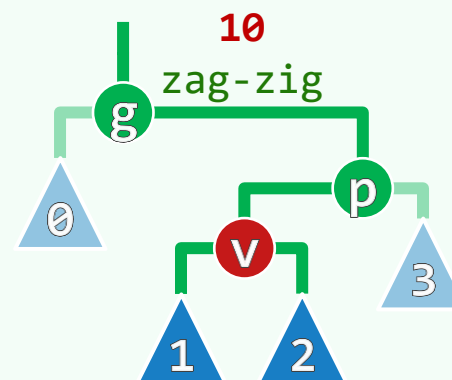
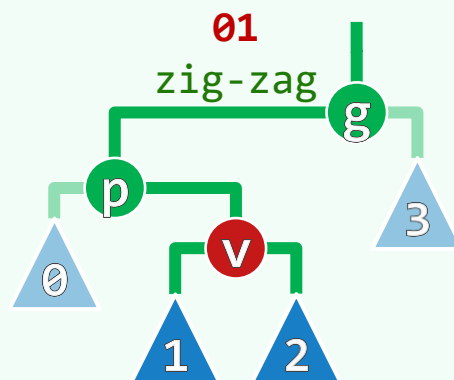
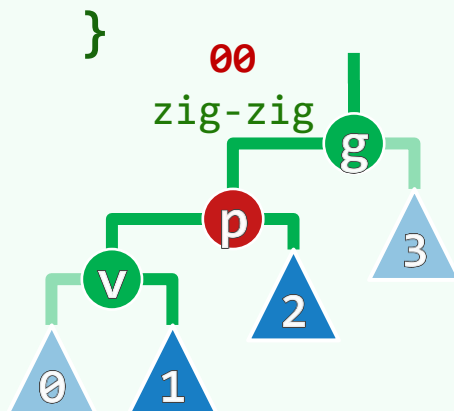
统一调整：总体

```
template<typename T> BNP<T> BST<T>::rotate( BNP<T> g ) {  
    BNP<T> p = tallerChild(g); int TurnP = (p == g->rc);  
    BNP<T> v = tallerChild(p); int TurnV = (v == p->rc);  
    BNP<T> r = ( TurnP == TurnV ) ? p : v; //子树新的根节点  
    ( FromParentTo(g) = r )->parent = g->parent; //须保持与母树的联接  
    switch ( ( TurnP << 1 ) | TurnV ) { /* 视p、v的拐向，无非四种情况 */ }  
}
```



统一调整：四种情况

```
switch ( ( TurnP << 1 ) | TurnV ) {  
    case 0b00 : return connect342( v, v->rc, p, p->rc, g );  
    case 0b01 : return connect342( p, v->lc, v, v->rc, g );  
    case 0b10 : return connect342( g, v->lc, v, v->rc, p );  
    default/*11*/: return connect342( g, p->lc, p, v->lc, v );  
}
```



AVL：综合评价

❖ 优点 无论查找、插入或删除，最坏情况下的复杂度均为 $\mathcal{O}(\log n)$

$\mathcal{O}(n)$ 的存储空间

❖ 缺点 借助高度或平衡因子，为此需改造元素结构，或额外封装

实测复杂度与理论值尚有差距

- 插入/删除后的旋转，成本不菲
- 删除操作后，最多需旋转 $\Omega(\log n)$ 次
- 若需频繁进行插入/删除操作，未免得不偿失

单次动态调整后，全树拓扑结构的变化量可能高达 $\Omega(\log n)$

❖ 有没有更好的结构呢？ //保持兴趣